Environment Setup Instructions

The original GraphNorm repository was designed for older CUDA-enabled Linux environments using outdated versions of PyTorch and DGL. Since the experiments in this project were conducted on an Apple Silicon MacBook using CPU execution, several dependency and compatibility updates were required.

1. Create Conda Environment

conda create -n graphnorm python=3.8 -c conda-forge
conda activate graphnorm

2. Install Core Dependencies

pip install torch==2.0.1
pip install dgl==1.1.3
pip install packaging
pip install setuptools==65.5.0

3. Modify requirements.txt

The original repository contained outdated package versions that are incompatible with modern Apple Silicon/macOS environments.

The following replacements were made:

Original Dependency	Updated Dependency
torch==1.4.0	torch==2.0.1
dgl_cu101==0.4.2	dgl==1.1.3
dgl==0.4.3.post2	dgl==1.1.3
numpy==1.17.4	numpy
scikit_learn==0.23.1	scikit-learn

After editing the file:

pip install -r requirements.txt

4. DGL API Compatibility Fixes

Several APIs used in the original codebase were deprecated in newer DGL versions. The following modifications were required:

Import Updates

from dgl.batched_graph import sum_nodes, mean_nodes, max_nodes

changed to:

from dgl.readout import sum_nodes, mean_nodes, max_nodes

and:

from dgl.graph import DGLGraph

changed to:

from dgl import DGLGraph

Edge Addition API

g.add_edge(u, v)

changed to:

g.add_edges(u, v)

Graph Size API

len(g)

changed to:

g.num_nodes()

Batch Node API

graph.batch_num_nodes

changed to:

graph.batch_num_nodes()

5. Tensor Conversion Fixes

Newer DGL versions require PyTorch tensors instead of NumPy arrays for node features.

Example modifications:

g.ndata['label'] = torch.tensor(nlabels)

and one-hot node features were generated using:

num_classes = 3

```
g.ndata['attr'] = torch.zeros(
    (g.num_nodes(), num_classes),
    dtype=torch.float32
)

g.ndata['attr'][torch.arange(g.num_nodes()), g.ndata['label'].long()] = 1
```

6. CPU Compatibility Fixes

The original repository assumed CUDA availability. Since experiments were executed on CPU, all `.cuda()` calls were modified to execute conditionally:

```
if args.gpu >= 0:
    tensor = tensor.cuda()
```

Training was executed using:

--gpu -1

7. Running the Experiments

Example command used for execution:

```
python train_graph_level.py \
  --gpu -1 \
  --dataset PROTEINS \
  --exp proteins_gin_gn_fast \
  --log_dir ../log/proteins_gin_gn_fast \
```

```
--data_dir ../data \
--model GIN \
--weight_decay 0.05 \
--norm_type gsgn \
--graph_pooling_type sum \
--self_loop \
--learn_eps \
--epoch 150 \
--fold_idx 0
```

8. Notes

- All experiments were executed on CPU mode.
- Training time on Apple Silicon CPU was approximately 0.6–0.7 seconds per epoch for the PROTEINS dataset.
- The repository required multiple compatibility patches due to significant API changes between older and modern DGL/PyTorch versions.